



**B.Sc. (General/Special) Degree
Programme
Faculty of Applied Sciences
University of Sri Jayewardenepura**

ICT 1022 2.0 Computer System Architecture

Presented By:

Surani Tissera(PhD)

Department of Computer Science

MIPS32 INSTRUCTION SET



This is a property of Department of Computer Science, Faculty of Applied Science, University of Sri Jayewardenepura.

Instruction Set Classification

MIPS32 instruction can be classified into the following functional groups:

- Load and Store
- Arithmetic and Logical
- Jump and Branch
- Miscellaneous
- Coprocessor instruction (to activate an auxiliary processor).

All instructions are encoded in 32 bits.



Alignment of Words in Memory

MIPS requires that all words must be aligned in memory to word boundaries.

- Must start from an address that is some power of 4.
- Last two bits of the address must be 00.

- Allows a word to be fetched in a single cycle.

- Misaligned words may require two cycles.

Address				
0000H	w1	w1	w1	w1
0004H		w2	w2	w2
0008H	w2			
000CH			w3	w3
0010H	w3	w3		
0014H				w4
0018H	w4	w4	w4	
w1 is aligned, but w2, w3, w4 are not				



(a) Load and Store Instructions

- MIPS32 is a load-store architecture.
 - All operations are performed on operands held in processor registers.
 - Main memory is accessed only through LOAD and STORE instructions.
- There are various types of LOAD and STORE instructions, each used for a particular purpose.
 - a) By specifying the size of the operand (W: word, H: half-word, B: byte)
 - Examples: LW, LH, LB, SW, SH, SB
 - b) By specifying whether the operand is signed (by default) or unsigned.



- Examples: LHU, LBU

(a) Load and Store Instructions

- c) Accessing fields that are not word aligned.
 - Examples: LWL, LWR, SWL, SWR
- d) Atomic memory update for read-modify-write instructions
 - Examples: LL, SC



Data sizes that can be accessed through LOAD and STORE

Data Size	Load Signed	Load Unsigned	Store
Byte	YES	YES	YES
Half-word	YES	YES	YES
Word	YES	Only for MIPS64	YES
Unaligned word	YES		YES
Linked word (atomic modify)	YES		YES



Data sizes that can be accessed through LOAD and STORE

Type	Mnemonic	Function
Aligned	LB	Load Byte
	LBU	Load Byte Unsigned
	LH	Load Half-word
	LHU	Load Half-word Unsigned
	LW	Load Word
	SB	Store Byte
	SH	Store Half-word
	SW	Store Word

Type	Mnemonic	Function
Unaligned	LWL	Load Word Left
	LWR	Load Word Right
	SWL	Store Word Left
	SWR	Store Word Right
Atomic Update	LL	Load Linked Word
	SB	Store Conditional Word



(b) Arithmetic and Logic Instructions

- All arithmetic and logic instructions operate on registers.
- Can be broadly classified into the following categories:
 - ALU immediate
 - ALU3-operand
 - ALU2-operand
 - Shift
 - Multiply and Divide



Type	Mnemonic	Function
16-bit Immediate Operand	ADDI	Add Immediate Word
	ADDIU	Add Immediate Unsigned Word
	ANDI	AND Immediate
	LUI	Load Upper Immediate
	ORI	OR Immediate
	SLTI	Set on Less Than Immediate
	SLTIU	Set on Less Than Immediate Unsigned
	XORI	Exclusive-OR Immediate



Type	Mnemonic	Function
3-Operand	ADD	Add Word
	ADDU	Add Unsigned Word
	AND	Logical AND
	NOR	Logical NOR
	SLT	Set on Less Than
	SLTU	Set on Less Than Unsigned
	SUB	Subtract Word
	SUBU	Subtract Unsigned Word
	XOR	Logical XOR



Type	Mnemonic	Function
2-Operand	CLO	Count Leading Ones in Word
	CLZ	Count Leading Zeros in Word
Type	Mnemonic	Function
Shift	ROTR	Rotate Word Right
	ROTRV	Rotate Word Right Variable
	SLL	Shift Word Left Logical
	SLLV	Shift Word Left Logical Variable
	SRA	Shift Word Right Arithmetic
	SRAV	Shift Word Right Arithmetic Variable
	SRL	Shift Word Right Logical
	SRLV	Shift Word Right Logical Variable



(c) Multiply and Divide Instructions

- The multiply and divide instructions produce twice as many result bits.
 - When two 32-bit numbers are multiplied, we get a 64-bit product.
 - After division, we get a 32-bit quotient and a 32-bit remainder.
- Results are produced in the HI and LO register pair.
 - a) For multiplication, the low half of the product is loaded into LO, while the higher half in HI.
 - b) Multiply-Add and Multiply-Subtract produce a 64-bit product, and adds or subtracts the product from the concatenated value of HI and LO.
 - c) Divide produces a quotient that is loaded into LO and a remainder that is loaded into HI.



(c) Multiply and Divide Instructions

Only exception is the MUL instruction, which delivers the lower half of the result directly to a GPR.

- Useful in situations where the product is expected to fit in 32 bits.



(c) Multiply and Divide Instructions

Type	Mnemonic	Function
Multiply and Divide	DIV	Divide Word
	DIVU	Divide Unsigned Word
	MADD	Multiply and Add Word
	MADDU	Multiply and Add Word Unsigned
	MFHI	Move from HI
	MFLO	Move from LO
	MSUB	Multiply and Subtract Word
	MSUBU	Multiply and Subtract Word Unsigned
	MTHI	Move to HI
	MTLO	Move to LO
	MUL	Multiply Word to Register
	MULT	Multiply Word
	MULTU	Multiply Unsigned Word



(d) Jump and Branch Instructions

The following types of Jump and Branch instructions are supported by MIPS32.

- PC relative conditional branch
- A 16-bit offset is added to PC.
 - PC-region unconditional jump
- A 28-bit offset if added to PC.
 - Absolute (register) unconditional jump
 - Special Jump instructions that link the return address in R31.



(d) Jump and Branch Instructions

Type	Mnemonic	Function
Unconditional Jump within a 256 MB Region	J	Jump
	JAL	Jump and Link
	JALX	Jump and Link Exchange

Type	Mnemonic	Function
Unconditional Jump using Absolute Address	JALR	Jump and Link Register
	JALRHB	Jump and Link Register with Hazard Barrier
	JR	Jump Register
	JRHB	Jump Register with Hazard Barrier



(d) Jump and Branch Instructions

Type	Mnemonic	Function
PC-Relative Conditional Branch Comparing Two Registers	BEQ	Branch on Equal
	BNE	Branch on Not Equal

Type	Mnemonic	Function
PC-Relative Conditional Branch Comparing With Zero	BGEZ	Branch on Greater Than or Equal to Zero
	BGEZAL	Branch on Greater Than or Equal to Zero and Link
	BGTZ	Branch on Greater than Zero
	BLEZ	Branch on Less Than or Equal to Zero



(e) Miscellaneous Instructions

These instructions are used for various specific machine control purposes.

- They include:
 - Exception instructions
 - Conditional MOVE instructions
 - Prefetch instructions
 - NOP instructions



(e) Miscellaneous Instructions

Type	Mnemonic	Function
System Call and Breakpoint	BREAK	Breakpoint
	SYSCALL	System Call
Type	Mnemonic	Function
Trap-on-Condition Comparing Two Registers	TEQ	Trap if Equal
	TGE	Trap if Greater Than or Equal
	TGEU	Trap if Greater Than or Equal Unsigned
	TLT	Trap if Less Than
	TLTU	Trap if Less Than Unsigned
	TNE	Trap if Not Equal

Type	Mnemonic	Function
Trap-on-Condition Comparing an Immediate Value	TEQI	Trap if Equal Immediate
	TGEI	Trap if Greater Than or Equal Immediate
	TGEIU	Trap if Greater Than or Equal Immediate Unsigned
	TLTI	Trap if Less Than Immediate
	TLTIU	Trap if Less Than Immediate Unsigned
	TNEI	Trap if Not Equal Immediate



(e) Miscellaneous Instructions

Type	Mnemonic	Function
Conditional Move	MOVF	Move Conditional on Floating Point False
	MOVN	Move Conditional on Not Zero
	MOVT	Move Conditional on Floating Point True
	MOVZ	Move Conditional on Zero

Type	Mnemonic	Function
Prefetch	PREF	Prefetch Register+Offset
NOP	NOP	No Operation

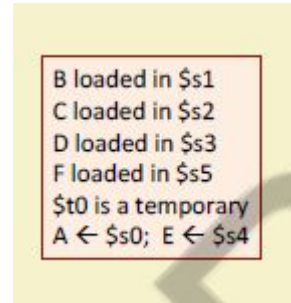
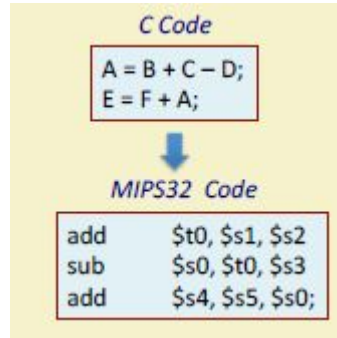
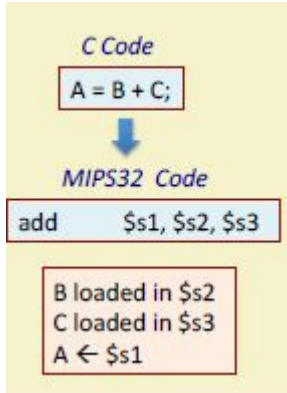


(e) Coprocessor Instructions

- The MIPS architecture defines four coprocessors (designated CP0, CP1, CP2, and CP3).
 - Coprocessor 0 (CP0) is incorporated on the CPU chip and supports the virtual memory system and exception handling. CP0 is also referred to as the System Control Coprocessor.
 - Coprocessor 1 (CP1) is reserved for the floating point coprocessor.
 - Coprocessor 2 (CP2) is available for specific implementations.
 - Coprocessor 3 (CP3) is available for future extensions.
- These instructions are not discussed here.



Some Examples of MIPS32 Arithmetic



Example on LOAD and STORE

C Code

```
A[10] = X - A[12];
```



MIPS32 Code

```
lw    $t0, 48($s3)
sub    $t0, $s2, $t0
sw    $t0, 40($s3);
```

\$s3 contains the starting
address of the array A

\$s2 loaded with X

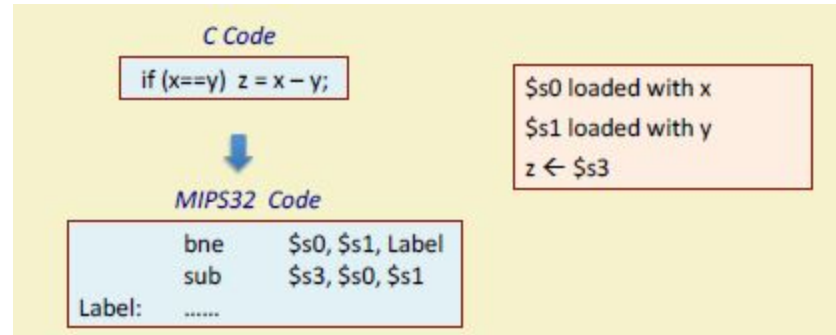
\$t0 is a temporary

Address of A[10] will be
\$s3+40 (4 bytes per element)

Address of A[12] will be
\$s3+48



Examples on Control Constructs



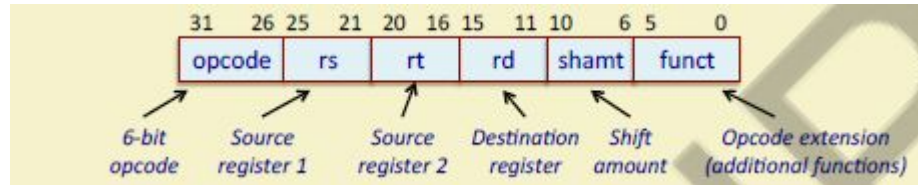
MIPS Instruction Encoding

- All MIPS32 instructions can be classified into three groups in terms of instruction encoding.
 - R-type (Register), I-type (Immediate), and J-type (Jump).
 - In an instruction encoding, the 32 bits of the instruction are divided into several fields of fixed widths.
 - All instructions may not use all the fields.
- Since the relative positions of some of the fields are same across instructions, instruction decoding becomes very simple.



(a) R-type Instruction Encoding

- Here an instruction can use up to three register operands.
 - Two source and one destination.
- In addition, for shift instructions, the number of bits to shift can also be specified.



(a) R-type Instruction Encoding

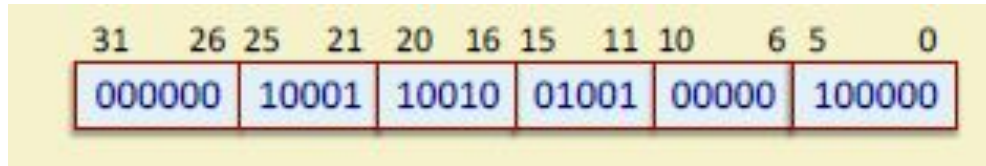
Examples of R-type instructions:

add \$s1, \$s2, \$s3

sub \$t1, \$s3, \$s4

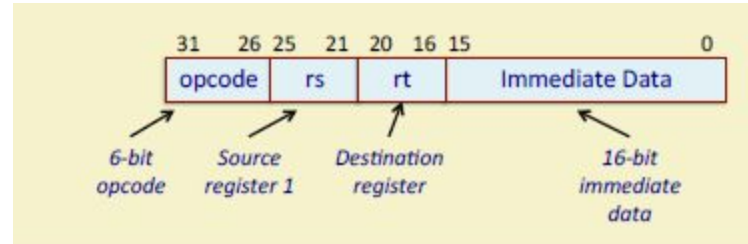
sla \$s1, \$s2, 5 // shift left \$s2 by 5 places, and store in \$s1

- An example instruction encoding: *add \$t1, \$s1, \$s2*
- Recall: \$t1 is R9, \$s1 is R17, and \$s2 is R18.
- For “add”, opcode = 000000, and funct = 100000,



(b) I-type Instruction Encoding

- Contains a 16-bit immediate data field.
- Supports one source and one destination register.



(b) I-type Instruction Encoding

- Examples of I-type instructions:

lw \$s1, 50(\$s5)

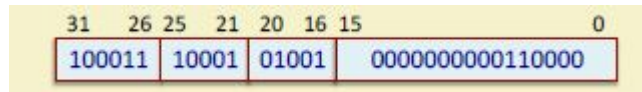
sw \$t1, 100(\$s1)

addi \$t0, \$s1, 188

beq \$s1, \$s2, Label // Label is encoded as a 16-bit offset relative to PC

bne \$s3, \$zero, Label

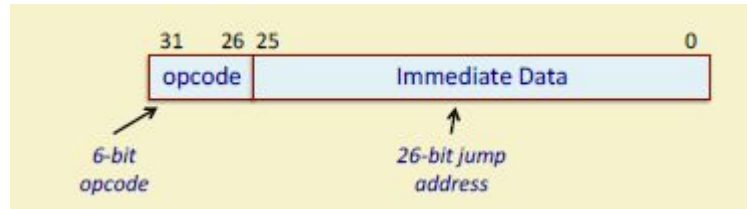
- An example instruction encoding: lw \$t1, 48(\$s1)
- Recall: \$t1 is R9, \$s1 is R17.
- For “lw”, opcode = 100011, and funct = 100000,



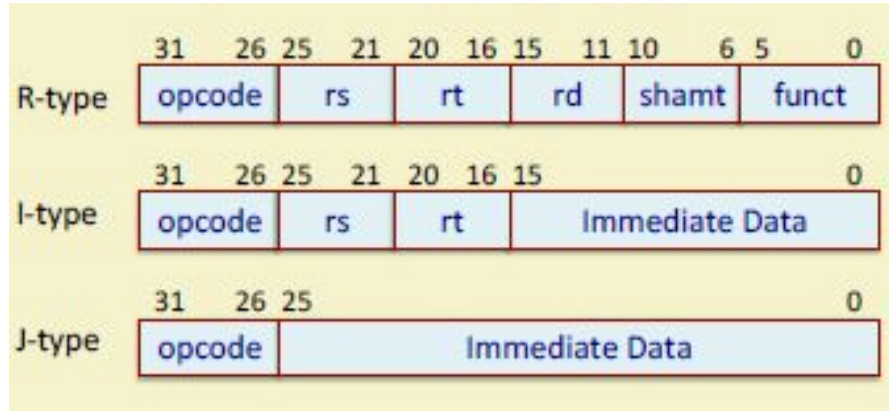
(c) J-type Instruction Encoding

- Contains a 26-bit jump address field.
 - Extended to 28 bits by padding two 0's on the right.

Example: *j Label*



A Quick View



- Some instructions require two register operands rs & rt as input, while some require only rs.
- Gets known only after instruction is decoded.
- While decoding is going on, we can prefetch the registers in parallel.
- May or may not be required later.

Similarly, the 16-bit and 26-bit immediate data are retrieved and sign-extended to 32-bits in case they are required later.



Addressing Modes in MIPS32

- Register addressing `add $s1, $s2, $s3`
- Immediate addressing `addi $s1, $s2, 200`
- Base addressing `lw $s1, 150($s2)`
 - Content of a register is added to a “base” value to get the operand address.
- PC relative addressing `beq $s1, $s2, Label`
 - 16-bit offset is added to PC to get the target address.
- Pseudo-direct addressing `j Label`
 - 26-bit offset is shifted left by 2 bits and then added to PC to get the target address.



Thank
you!

dreamstime

